

Personalized Learning Path Generator

Full technical documentation — architecture, the adaptive engine, the ML models, API reference, and database setup.

Contents

1. Overview & Quickstart
2. Architecture
3. The Adaptive Engine
4. ML Models
5. API Reference
6. Database

1. Overview & Quickstart

Personalized Learning Path Generator

An adaptive Class 8 Mathematics practice tool. It diagnoses a student's level, adapts question difficulty in real time using a trained ML model, tracks per-topic mastery with an Elo-style algorithm, and gives teachers a live dashboard — a heatmap, ML-derived learning archetypes, and per-student drill-down.

Built as a hackathon prototype: one full-stack Next.js app (no separate backend service), SQLite-simple-to-Postgres database via Prisma, and a small set of genuinely trained (not hand-coded) ML models whose weights are exported as plain JSON and read at runtime with zero Python dependency.

Full documentation lives in [docs/](#):

Doc	Covers
docs/ARCHITECTURE.md	How the pieces fit together, folder structure, request flow
docs/ADAPTIVE-ENGINE.md	The mastery-tracking + question-selection algorithm, line by line
docs/ML-MODELS.md	All 5 trained models: what they predict, training data, accuracy, how to retrain
docs/API.md	Every API route: method, params, request/response shape
docs/DATABASE.md	Prisma schema, Postgres setup, migrations

Quickstart

```
# 1. Install dependencies
npm install

# 2. Setup environment variables
cp .env.example .env

# 3. Start database (Choose Option A or Option B):
# Option A (Easiest - Docker):
docker compose up -d

# Option B (Local Mac Postgres):
# brew services start postgresql
# createdb erodehack
# (ensure DATABASE_URL in .env matches: postgresql://<your-username>@localhost:5432/erodehack)

# 4. Apply schema and seed initial questions
npx prisma db push
npm run seed          # loads 4 topics x 10 questions

# 5. Start dev server
npm run dev           # http://localhost:3000 (or 3001)
```

Requires a running Postgres instance and `DATABASE_URL` set in `.env` (see [docs/DATABASE.md](#) for detailed credentials setup & troubleshooting). No API keys required — there is no external LLM call in this build.

To populate the teacher dashboard with realistic-looking data before a demo:

```
BASE_URL=http://localhost:3000 npx tsx scripts/simulate-demo-data.ts
```

To retrain the ML models (optional — trained weights are already committed to `lib/models/*.json`):

```
python3 -m venv venv && source venv/bin/activate
pip install scikit-learn numpy
python scripts/train_difficulty_model.py
python scripts/train_cluster_model.py
python scripts/train_time_to_mastery_model.py
python scripts/train_topic_priority_model.py
python scripts/train_at_risk_model.py
```

Tech stack

- **Next.js 16** (App Router) + TypeScript
- **Tailwind CSS 4**
- **Prisma 6** + **PostgreSQL** (local, via Homebrew)
- **Recharts** (installed; the dashboard heatmap is hand-rolled HTML/CSS per accessibility guidance — see [docs/ARCHITECTURE.md](#))
- **scikit-learn** (Python, training-time only) — logistic/linear regression and KMeans, exported to JSON for zero-dependency TypeScript inference

What's genuinely "ML" here vs. hand-coded

Two things are commonly hand-coded in prototypes like this and were instead

trained on data (synthetic, since there's no real student data yet):

1. **Question difficulty selection** — a logistic regression, not an if/else ladder. See [docs/ML-MODELS.md](#).
2. **Student learning archetypes** on the teacher dashboard — a KMeans clustering model, not a hardcoded rule. See [docs/ML-MODELS.md](#).

Three more models are wired in alongside these: time-to-mastery estimation (`/revision`'s "attempts to mastery" estimate), class-wide topic-priority ranking (`/teacher/class-insights`'s Class Focus panel), and an at-risk/declining-trend detector (`/teacher`'s heatmap flag). See [docs/ML-MODELS.md](#) for the full picture, including honest accuracy numbers and where each model's fit is genuinely strong vs. weak.

2. Architecture

Why one Next.js app, not separate services

The scope decision from day one: build this as **one full-stack Next.js (App Router) application** in TypeScript, not separate frontend/backend services. The "adaptive engine" and "ML inference" logic live in plain TypeScript modules under `lib/`, called directly from Next.js API routes. This keeps the whole thing runnable with one `npm run dev` and trivially deployable, while still being accurate to describe as "an adaptive engine with a trained ML model" — it genuinely is one, just not a microservice.

Folder structure

app/	
page.tsx	Landing page (role picker: Student / Teacher)
practice/page.tsx	Student practice UI (diagnostic + adaptive), formerly quiz/page.tsx
revision/page.tsx	Student's weak-topic list + "Practice this" + time-to-mastery estimate (Model 3)
dashboard/page.tsx	Student's own home dashboard (progress rings, gamification bar, shortcuts) – NOT the teacher view, despite the name
profile/page.tsx	Student profile page
progress/page.tsx	Student progress page
teacher/page.tsx	Teacher class dashboard: heatmap + archetype donut + declining-trend badges (Model 5)
teacher/class-insights/page.tsx	Cohort diagnostics + Class Focus panel (Model 4) + persona distribution
teacher/resources/page.tsx	Question bank management (create/view questions per topic)
teacher/students/[id]/page.tsx	Per-student drill-down for teachers
api/	
student/route.ts	POST create-or-find student (case-insensitive name match), GET list students
student/[id]/summary/route.ts	GET one student's masteries, weak topics, attempts, and generated insights – powers the student dashboard/profile
dashboard/route.ts	GET all students shaped for the teacher heatmap: archetype (Model 2), at-risk topics (Model 5), class-wide topic priority ranking (Model 4)
dashboard/student/[id]/route.ts	GET one student's detail for the teacher drill-down view
quiz/next/route.ts	GET next question (diagnostic or adaptive)
quiz/answer/route.ts	POST an answer, updates Attempt + Mastery
revision/route.ts	GET a student's weak topics with attempts-to-mastery estimate (Model 3)
questions/route.ts	GET/POST the question bank (teacher resources page)
lib/	
prisma.ts	Prisma client singleton
session.ts	Client-side localStorage session (no auth)
colors.ts	Mastery color-band mapping (status palette)
brand.ts	Shared copy constants (e.g. CLASS_LABEL)
archetype.ts	Builds the fixed-order mastery vector and

student-data.ts	calls Model 2's assignArchetype() Shared per-student query helpers (mastery, weak topics, attempt counts) used by multiple API routes
insights.ts	Generates plain-English insight strings for the student summary endpoint – also reuses Model 5's estimateTrend() helper
gamification.ts	Streaks/XP/badge logic for GamificationBar
explanations.ts	Hand-written per-topic hint/step-by-step text shown after an incorrect answer (not ML – a lookup table, unlike selectDifficulty)
topic-icons.ts	Emoji icon per topic name, for UI badges
adaptive-engine.ts	Core algorithm: pure functions, no DB access
quiz-service.ts	DB-aware layer: wraps adaptive-engine with Prisma queries, handles the diagnostic phase
ml/	
difficulty-model.ts	Inference for Model 1 (wired into engine)
cluster-model.ts	Inference for Model 2 (wired into /teacher's heatmap)
time-to-mastery-model.ts	Inference for Model 3 (wired into /revision)
topic-priority-model.ts	Inference for Model 4 (wired into /teacher/class-insights)
at-risk-model.ts	Inference for Model 5 (wired into /teacher's declining-trend badge)
models/	
*.json	Trained weights, one file per model
prisma/	
schema.prisma	Data model (Student, Topic, Question, Attempt, Mastery)
seed.ts	Loads the 4 topics x 10 questions, idempotent per topic (safe to re-run)
migrations/	SQL migration history
scripts/	
train_*.py	Python: generates synthetic data, trains a model, writes lib/models/*.json + data/*.csv
test-*.ts	TypeScript: sanity-checks each ml/*.ts layer
test-engine.ts	Simulates 30 attempts through the adaptive engine, prints the mastery trajectory
simulate-demo-data.ts	Plays fake students through the real engine (via Prisma directly) to seed a realistic-looking teacher dashboard
data/	
*.csv	The synthetic training data each Python script generates (200k rows each)
components/	
Sidebar.tsx	Student/teacher nav shell
MasteryRing.tsx	Per-topic/overall mastery ring (student dashboard), successor to MasteryBar
ArchetypeDonut.tsx	Archetype distribution chart (Model 2)
GamificationBar.tsx	Streaks/XP/badges strip
Confetti.tsx, GrowthIllustration.tsx, Logo.tsx, PitchDeckModal.tsx, Scratchpad.tsx	Supporting UI

Request flow: a student answers a question

```

Browser (app/practice/page.tsx)
-> GET /api/quiz/next?studentId=...
    app/api/quiz/next/route.ts
        -> lib/quiz-service.ts: getNextQuestion()
            - counts the student's total attempts
            - if < topics.length x 2: diagnostic phase (see ADAPTIVE-ENGINE.md)
            - else: adaptive phase
                -> lib/adaptive-engine.ts: selectTargetTopic()
                    - weighted interleaved sampling over unmastered
                      topics, not pure greedy weakest-first
                -> lib/ml/difficulty-model.ts: selectDifficulty()
                    (Model 1 – logistic regression inference)
                -> lib/adaptive-engine.ts: selectQuestion()
            <- { question, topic, reason, mode, questionNumber, totalDiagnostic }
        -> student picks an option
    -> POST /api/quiz/answer { studentId, questionId, selectedIdx }
        app/api/quiz/answer/route.ts
            - looks up the question's real answerIdx server-side (never sent
              to the client) to determine correct/incorrect
            - lib/adaptive-engine.ts: updateMastery() – Elo-style update
            - upserts the Mastery row in Postgres
            <- { correct, correctIdx, priorScore, newScore, scoreDelta,
                topicName, updatedMastery }
        -> mastery bar animates, feedback + reason shown

```

Request flow: teacher dashboard

```

Browser (app/teacher/page.tsx, app/teacher/class-insights/page.tsx)
-> GET /api/dashboard
    app/api/dashboard/route.ts
        - fetches all students with their Mastery rows and Attempt history
        - builds a 4-number mastery vector per student, in the FIXED
          topic order the cluster model was trained on (Fractions,
          Ratios, Linear Equations, Percentages) – order matters here,
          since it must match training time exactly
        -> lib/ml/cluster-model.ts: assignArchetype()
            (Model 2 – nearest-centroid KMeans inference)
        - for each student x topic: lib/ml/at-risk-model.ts estimateTrend()
          + isAtRisk() over their last 10 attempts on that topic
            (Model 5 – declining-trend detector)
        - aggregates class-wide avgMastery / pctStruggling / stdMastery
          per topic, then lib/ml/topic-priority-model.ts rankTopicsByPriority()
            (Model 4 – class-wide instructional priority)
        <- { students: [{ archetype, topicMastery, atRiskTopics, isAtRisk }],
            classAverage, atRiskCount, needsAttentionCount, decliningCount,
            topicPriority }
    -> /teacher renders the heatmap (color-coded cells, archetype badge,
        declining-trend flag per student)
    -> /teacher/class-insights renders the Class Focus panel from topicPriority
    -> clicking a student -> GET /api/dashboard/student/[id] for the teacher
        drill-down (a separate endpoint from the student's own summary)

```

No authentication, by design

This is a demo prototype: the landing page is a hardcoded role picker (Student / Teacher), no passwords. A student's identity is just a `cuid()` stored in `localStorage` (`lib/session.ts`) — there's no session/JWT/cookie layer. Do not use this auth pattern outside a demo context.

Database: SQLite -> Postgres

The build started on SQLite (Prisma's zero-setup option for a fast start), then was migrated to a local PostgreSQL instance (via Homebrew) so the data could be inspected in DBeaver like a normal relational database. See [DATABASE.md](#) for the full schema and connection details. The switch only touched `prisma/schema.prisma`'s `datasource` block and `DATABASE_URL` — application code is unaware of which database engine is behind Prisma.

Why the heatmap isn't a Recharts component

Recharts is installed (per the original tech-stack spec) but has no native heatmap primitive. The teacher dashboard's topic x student grid is hand-rolled as a colored HTML table instead, following a dataviz accessibility pass: cells use a fixed 3-step status palette (red/amber/green) with the percentage always printed as visible text inside the cell — so identity/severity is never color-alone, and the page works for colorblind viewers without relying on hue discrimination.

3. The Adaptive Engine

Source: `lib/adaptive-engine.ts` (pure functions, no database access — fully unit-testable) and `lib/quiz-service.ts` (the DB-aware layer that calls it).

1. Mastery tracking (Elo-inspired)

Every student has a `Mastery` row per topic: a float in `[0.05, 0.95]` starting at `0.5` (assumed 50/50 before any evidence). After every answered question, `updateMastery()` nudges it toward the observed outcome:

```
// lib/adaptive-engine.ts
const LEARNING_RATE = 0.15; // "K" — how fast mastery reacts to new evidence

function updateMastery(currentMastery: number, correct: boolean): number {
  const expected = currentMastery; // treat current score as P(correct)
  const actual = correct ? 1 : 0;
  const next = currentMastery + LEARNING_RATE * (actual - expected);
  return clamp(next, 0.05, 0.95); // never fully certain either way
}
```

This is the same shape as an Elo rating update: the score moves toward whatever just happened, by an amount proportional to how surprising the outcome was (a correct answer from someone with `mastery=0.9` barely moves the needle; a correct answer from someone at `mastery=0.1` moves it a lot).

The clamp to `[0.05, 0.95]` matters for two reasons: it keeps the score usable as a probability-like input to the ML difficulty model (which can't handle exactly 0 or 1), and it means no single answer can ever fully "prove" mastery or its absence — consistent with treating this as a noisy signal, not ground truth.

2. Diagnostic phase (first 8 questions)

Before there's any evidence, the app can't yet tell which topic is weak. So the first `topics.length x 2 = 8` questions cycle through each of the 4 topics twice, always at difficulty 2 (the "medium" band), to seed a rough mastery baseline for every topic before switching to adaptive mode. This logic lives in `getNextQuestion()` in `lib/quiz-service.ts`:

```
const topicOrder = [Fractions, Ratios, LinearEq, Percentages, // round 1
                    Fractions, Ratios, LinearEq, Percentages]; // round 2
const targetTopicId = topicOrder[totalAttemptsSoFar];
```

It picks a difficulty-2 question in that topic the student hasn't already been asked, falling back to "any difficulty-2 question in that topic" if they've somehow exhausted the unseen ones.

3. Adaptive phase (after the diagnostic)

Once 8+ attempts exist, every subsequent question goes through `selectTargetTopic()` then `selectQuestion()`.

Picking the topic: weighted interleaved sampling

```
function selectTargetTopic(masteries, restrictToTopicId?) {
  if (restrictToTopicId) {
```

```

    // "Practice this" mode: strictly serve that one topic
    ...
}

const pool = masteries.filter(m => m.score < MASTERY_THRESHOLD); // 0.8 — not yet mastered
const activePool = pool.length > 0 ? pool : masteries;           // fallback: everything mastered

// Interleaved weighted sampling: lower mastery gets higher probability
// weight = (1.05 - score) ^ 2
const weights = activePool.map(m => Math.pow(Math.max(0.08, 1.05 - m.score), 2));
const totalWeight = weights.reduce((sum, w) => sum + w, 0);

let randomVal = Math.random() * totalWeight;
let selected = activePool[0];
for (let i = 0; i < activePool.length; i++) {
    randomVal -= weights[i];
    if (randomVal <= 0) { selected = activePool[i]; break; }
}
...
}

```

Among topics not yet "mastered" (`score < 0.8`), the topic isn't always the single weakest one — instead each unmastered topic gets a probability weight of $(1.05 - \text{score})^2$, so a topic at 20% mastery is far more likely to be picked than one at 60%, but the stronger topic still gets served sometimes. This is a deliberate move away from a pure greedy weakest-first strategy (which would hammer one topic for many consecutive questions): weighted sampling still concentrates practice on the weakest areas but interleaves other still-unmastered topics, which the spaced/ interleaved-practice literature generally finds aids retention better than one long uninterrupted block on a single topic.

The returned `reason` string reflects which case happened: if the sampled topic is the actual weakest in the pool it reads as **"Targeted Remediation"**; if a different (but still unmastered) topic was sampled for interleaving, it reads as **"Interleaved Reinforcement"** — see below.

If `restrictToTopicId` is set (the `/revision` page's "Practice this" button, via `?topic=<id>` on the practice page URL), the weighted sampling is skipped entirely and every question comes from that one topic.

Picking the difficulty: the ML model, not a hand-coded ladder

```
const targetDifficulty = selectDifficulty(weakest.score); // lib/ml/difficulty-model.ts
```

This calls **Model 1** (logistic regression — see [ML-MODELS.md](#)), which was trained to predict `P(correct | mastery, difficulty)` and picks whichever of difficulty 1/2/3 puts the student closest to a 75% success rate — the "desirable difficulty" zone from learning science. This replaced an earlier hand-coded `if (score < 0.4) return 1; ...` ladder.

The explainability string

Every returned question comes with a plain-English `reason`, one of three templates depending on which branch of `selectTargetTopic()` produced it:

```

"Targeted Remediation: Your Ratios mastery (22%) is your priority improvement area."
"Interleaved Reinforcement: Practicing Ratios (61% mastery) alongside your priority chapters."
"Focused Practice: Ratios (Current Mastery: 22%)." // "Practice this" / restrictToTopicId mode

```

This is a template string (not an LLM call — an earlier plan considered calling Claude per-question for this, but that was never wired in; see the "What's genuinely ML" note in the root README). It's shown on the student practice UI as a named feature so the reasoning behind each question is never opaque.

Picking the specific question: no-repeat rule

```
function selectQuestion(questions, targetTopicId, targetDifficulty, recentAttempts) {
  const recentOnTopic = recentAttempts.filter(a => a.topicId === targetTopicId).slice(-5);
  const recentlyCorrectIds = new Set(recentOnTopic.filter(a => a.correct).map(a => a.questionId));
  const eligible = topicQuestions.filter(q =>
    q.difficulty === targetDifficulty && !recentlyCorrectIds.has(q.id)
  );
  ...
}
```

Within the target topic and difficulty, a question already answered

correctly in the last 5 attempts on that topic is excluded (so a student doesn't just see the same easy win repeatedly). If that leaves nothing, it progressively relaxes: same difficulty regardless of recency, then any difficulty not recently seen, then anything at all — the app should never hard-stop just because the no-repeat rule ran out of room in a 10-question topic pool.

4. Weak-topic / revision plan

```
const WEAK_THRESHOLD = 0.5;
const WEAK_MIN_ATTEMPTS = 3;

function getWeakTopics(masteries, attemptCounts) {
  return masteries
    .filter(m => m.score < WEAK_THRESHOLD && attemptCounts[m.topicId] >= WEAK_MIN_ATTEMPTS)
    .sort((a, b) => a.score - b.score);
}
```

A topic only counts as "weak" (and shows up on */revision*) once there's been at least 3 attempts on it — otherwise a single unlucky first answer would immediately flag a topic the student hasn't really been tested on yet. Sorted weakest-first so the most urgent topic is at the top.

Testing this in isolation

scripts/test-engine.ts simulates 30 attempts for a fake student with a known skill profile per topic (e.g. 80% success on Fractions, 20% on Ratios) and prints the mastery trajectory round by round — useful for eyeballing that the engine converges sensibly without needing the database or the UI at all:

```
npx tsx scripts/test-engine.ts
```

4. ML Models

Five models were trained. All follow the same pattern: a Python script generates synthetic data (there's no real student data yet — this is a fresh prototype), trains a small scikit-learn model, evaluates it on a held-out 20% test split, and exports the learned weights as JSON. A TypeScript module then re-implements the model's math by hand (a sigmoid, a dot product, a nearest-centroid search) so the Next.js app can run inference with **zero Python dependency at runtime** — Python is only needed to retrain.

#	Model	Type	Wired into the app?
1	Difficulty selection	Logistic regression	Yes: lib/adaptive-engine.ts
2	Archetype clustering	KMeans	Yes: /teacher (class heatmap)
3	Time-to-mastery	Linear regression (quadratic features)	Yes: /revision
4	Topic priority	Linear regression	Yes: /teacher/class-insights
5	At-risk detector	Logistic regression	Yes: /teacher (declining-trend badge)

All five report **held-out accuracy in the 85-90% range** (see each section for exact numbers). Where the ground-truth generating formula is known, the script also computes the **Bayes-optimal ceiling** — the best any model could possibly score given the label noise baked into the data — so the reported accuracy is interpretable rather than a bare number.

Model 1 — Difficulty selection

File: `scripts/train_difficulty_model.py -> lib/models/difficulty-model.json -> lib/ml/difficulty-model.ts`

Predicts: `P(correct | mastery, difficulty)` — used to pick whichever difficulty (1/2/3) puts a student closest to a 75% success rate (the "desirable difficulty" zone).

Training data (`data/difficulty_training_data.csv`, 200,000 rows, columns `mastery`, `difficulty`, `correct`): for each row, `mastery` is uniform on [0,1], `difficulty` is uniform on {1,2,3}, and the ground truth is `p = sigmoid(11 * (mastery - 0.2 * difficulty))`, with `correct` sampled as a coin flip from that probability. The **11** steepness constant controls how much overlap/noise exists between classes — it's the main knob used to land held-out accuracy in the 85-90% band.

Held-out results (20% test split, 40,000 rows):

Metric	Value
Test accuracy	87.8%
Test ROC AUC	0.95

Metric	Value
Bayes-optimal ceiling	87.8%

The model's accuracy essentially equals the Bayes ceiling — it has fully recovered the true relationship; the remaining "error" is pure label noise that no model could avoid.

Learned weights: **mastery: +11.0, difficulty: -2.2** — signs asserted in the training script (mastery coefficient must be positive, difficulty must be negative) before the model is allowed to export.

TypeScript inference:

```
// lib/ml/difficulty-model.ts
export function predictCorrectProb(mastery: number, difficulty: number): number {
  const z = weights.mastery * mastery + weights.difficulty * difficulty + bias;
  return 1 / (1 + Math.exp(-z));
}

export function selectDifficulty(mastery: number): 1 | 2 | 3 {
  // returns whichever of {1,2,3} gives predictCorrectProb closest to 0.75
}
```

Model 2 — Student archetype clustering

File: `scripts/train_cluster_model.py -> lib/models/cluster-centroids.json -> lib/ml/cluster-model.ts`

Predicts: which of 3 learning archetypes a student's 4-topic mastery vector belongs to — "On Track", "Needs Support", or "Uneven — targeted help needed" — shown as a color-coded badge next to each student on the teacher dashboard.

Training data (`data/cluster_training_data.csv`, ~200,000 rows, columns `pattern`, `fractions`, `ratios`, `linear_equations`, `percentages`): three generating patterns, ~66,666 rows each —

- **uniformly strong:** all 4 topics ~0.8 + Gaussian noise
- **uniformly weak:** all 4 topics ~0.25 + Gaussian noise
- **spiky:** Fractions & Linear Equations ~0.8, Ratios & Percentages ~0.25 + Gaussian noise (a *fixed* high/low split — see the bug note below)

Noise std dev is **0.30**, tuned so the archetypes overlap enough to land held-out accuracy in the 85-90% range instead of a suspiciously perfect 100%.

A real bug found during this build: the "spiky" pattern originally

randomized which two of the four topics were high per student (matching the more general spec of "2 high, 2 low, randomize which 2"). That worked by luck at a small sample size (51 students), but at 200,000 rows the randomization averaged out — every dimension converged to ~0.5 across the whole "spiky" population, and the cluster centroid lost the unevenness signal entirely (KMeans found 2 near-identical mid-level clusters instead of 3 distinct archetypes, and the `assert len(set(labels)) == 3` check in the script failed). Fixed by using a **consistent** high/low topic split for the spiky pattern, which keeps the archetype statistically recoverable at any

dataset size. See the comment in `train_cluster_model.py` for the full explanation.

Held-out results (20% test split, ground truth = the generating pattern, not the KMeans label):

Metric	Value
Test clustering accuracy	86.5%
Silhouette score (test sample)	0.31

Clustering accuracy is measured by: fit KMeans on the train split, map each resulting cluster to its majority true pattern, then check how often a held-out point's assigned cluster matches its actual generating pattern.

Learned centroids (order: Fractions, Ratios, Linear Equations, Percentages — **this exact order matters**, since `app/api/dashboard/route.ts` builds each student's mastery vector in this fixed order to match):

Label	Centroid
Needs Support	<code>[0.26, 0.28, 0.26, 0.28]</code>
On Track	<code>[0.75, 0.77, 0.75, 0.77]</code>
Uneven — targeted help needed	<code>[0.77, 0.26, 0.78, 0.26]</code>

TypeScript inference: Euclidean distance from the input vector to each of the 3 centroids; returns the nearest one's label.

Model 3 — Time-to-mastery estimation

File: `scripts/train_time_to_mastery_model.py` -> `lib/models/time-to-mastery-model.json` -> `lib/ml/time-to-mastery-model.ts`

Predicts: how many practice attempts a student needs to reach mastery (`score >= 0.8`) on a topic, given their current mastery and an "aptitude" estimate (their steady-state per-attempt success probability).

Wired into the app: `app/api/revision/route.ts` calls `predictAttemptsToMastery(mastery, aptitude)` for each of a student's weak topics, where `aptitude` is the student's recent accuracy on that topic (falling back to their current mastery score if they have no attempts on it yet). The result is shown on `/revision` as "~N questions to mastery" next to each weak topic.

Training data (`data/time_to_mastery_training_data.csv`, 200,000 rows, columns `initial_mastery`, `aptitude`, `attempts_to_mastery`): this is the one model whose target isn't a formula — it's generated by **actually simulating** the real `updateMastery()` recurrence from `lib/adaptive-engine.ts` (`K = 0.15`) forward, attempt by attempt, capped at 150 attempts (rows that never reach 0.8 within the cap are "censored" at 150). For each of 200,000 (`initial_mastery`, `aptitude`) pairs, **10 independent trials are simulated and averaged** — a single trial is extremely noisy (attempts-to-mastery is a near-geometric random variable), so averaging a handful of replicates per student gives a far more stable regression target, the same way you'd average repeated measurements of any noisy quantity.

Why quadratic features were necessary: a plain linear fit on the raw (`initial_mastery`, `aptitude`) pair capped out around $R^2 = 0.40$ no matter how much replicate-averaging reduced the noise — because the true relationship is genuinely curved (attempts blow up as aptitude approaches the 0.8 target from above) and a straight line architecturally can't capture that. Adding `mastery2`, `aptitude2`, `mastery × aptitude` as extra features (still a *linear regression*, just over an expanded feature set) fixed this:

```
// lib/ml/time-to-mastery-model.ts
const raw =
  w.mastery * mastery + w.aptitude * aptitude +
  w.masterySq * mastery**2 + w.aptitudeSq * aptitude**2 +
  w.masteryAptitude * mastery * aptitude + bias;
return Math.round(clamp(raw, 1, 150));
```

Held-out results (20% test split, 40,000 rows):

Metric	Value
Test R^2	0.894
Test MAE	6.1 attempts

(For comparison: the naive 2-feature linear fit without replicate averaging or quadratic features scored $R^2 = 0.40$, MAE = 20.2 attempts — this is the one model in the set that genuinely needed feature engineering to hit the target range, not just a noise-level tuning knob.)

Model 4 — Class-wide topic priority recommender

File: `scripts/train_topic_priority_model.py` -> `lib/models/topic-priority-model.json` -> `lib/ml/topic-priority-model.ts`

Predicts: a 0-100 urgency score for "how much should the teacher focus whole-class instruction on this topic right now", from three class-wide stats: average mastery, fraction of students struggling (`mastery < 0.5`), and the spread (std dev) of mastery across the class.

Wired into the app: `app/api/dashboard/route.ts` computes each topic's `avgMastery`, `pctStruggling`, and `stdMastery` across all students and calls `rankTopicsByPriority()`, returned as `topicPriority` in the dashboard response. `/teacher/class-insights` renders it as the "Class Focus" panel — topics ranked highest-priority first with a progress bar for the 0-100 score.

Training data (`data/topic_priority_training_data.csv`, 200,000 rows, columns `avg_mastery`, `pct_struggling`, `std_mastery`, `priority`): ground truth is `100 * sigmoid(6*(0.5 - avg_mastery) + 3*pct_struggling + 4*std_mastery)` plus Gaussian noise (std 4, tuned for the target R^2).

Held-out results:

Metric	Value
Test R^2	0.859
Test MAE	10.2 priority points (0-100 scale)

Learned weights: `avg_mastery: -86.1, pct_struggling: +18.9, std_mastery: +43.0` — signs match the design intent (low mastery, more struggling students, and more unevenness all raise priority).

Model 5 — At-risk / declining-trend detector

File: `scripts/train_at_risk_model.py -> lib/models/at-risk-model.json -> lib/ml/at-risk-model.ts`

Predicts: whether a student is "at risk" on a topic — distinct from the static weak-topic rule already in the engine (`mastery < 0.5`) because this one also factors in **recent trend** (is their accuracy improving or declining lately), not just the current level. A student sitting at 0.55 mastery but trending sharply downward can be flagged before they cross the static threshold.

Wired into the app: `app/api/dashboard/route.ts` computes, per student per topic, `estimateTrend()` over that student's last 10 attempts on the topic, then calls `isAtRisk(mastery, trend)`. Topics flagged at-risk are returned as `atRiskTopics` per student; a student with any at-risk topic gets `isAtRisk: true`. The teacher dashboard (`/teacher`) shows this as a small flag badge next to the student's name in the heatmap (tooltip lists the affected topics) and rolls it up into a "Declining trend" stat card — distinct from, and complementary to, the static **Needs Support** / **Uneven** archetype badges from Model 2, since a student can look fine on the static mastery snapshot while trending down.

Training data (`data/at_risk_training_data.csv`, 200,000 rows, columns `mastery, trend, attempts_count, at_risk`): ground truth is `sigmoid(1.5 * (-6*(mastery-0.4) - 10*trend))`, `trend` uniform on `[-0.3, 0.3]`. The `1.5` steepness multiplier is the noise-tuning knob.

Held-out results:

Metric	Value
Test accuracy	87.2%
Test ROC AUC	0.95
Bayes-optimal ceiling	87.2%

Again essentially at the theoretical ceiling.

`estimateTrend()` **helper** (`lib/ml/at-risk-model.ts`): since the app doesn't store a mastery time series (only discrete **Attempt** rows), trend is estimated from a sequence of recent correct/incorrect outcomes as

`*(accuracy of the newer half) - (accuracy of the older half)*`. Returns 0 (neutral) with fewer than 4 attempts — not enough signal either way.

Retraining

Every `train_*.py` script is self-contained: it regenerates its CSV in `data/`, retrains, re-evaluates on a fresh 80/20 split, and overwrites its JSON in `lib/models/`. All five use `np.random.default_rng(42)`, so reruns are **deterministic** — rerunning without changing any script produces byte-identical output.

```
source venv/bin/activate
python scripts/train_<name>_model.py
```

After retraining, re-run the matching TypeScript sanity test (`npx tsx scripts/test-<name>-model.ts`) to confirm the new weights still pass the sign/ordering checks before trusting them in the app.

5. API Reference

All routes are Next.js App Router route handlers under `app/api/`. There is no authentication — `studentId` is a plain `cuid()` the client holds in `localStorage` (see `lib/session.ts`) and passes explicitly.

POST `/api/student`

Creates a student (or returns an existing one with a case-insensitive matching name — the most-active match if there are duplicates) and seeds a `Mastery` row (score `0.5`) for every topic on first creation.

File: `app/api/student/route.ts`

Request body:

```
{ "name": "Priya Sharma" }
```

Response 200:

```
{ "id": "cuid...", "name": "Priya Sharma" }
```

Response 400: `{ "error": "Name is required" }` if `name` is empty/missing.

GET `/api/student`

Lists all students, alphabetically by name.

Response 200:

```
{ "students": [{ "id": "...", "name": "..." }, ...] }
```

GET `/api/student/[id]/summary`

One student's masteries, overall progress, and recent attempts — powers the student's own `/dashboard` home page.

File: `app/api/student/[id]/summary/route.ts`

Response 200:

```
{
  "name": "Priya Sharma",
  "className": "Class 8 · Mathematics",
  "overallMastery": 0.63,
  "topicMastery": [
    { "topicId": "...", "topicName": "Fractions", "score": 0.63 },
    ...
  ],
  "hasNextQuestion": true,
  "weakTopicCount": 1,
}
```

```

    "totalAttempts": 42,
    "accuracy": 71,
    "recentAttempts": [
      {
        "id": "...", "questionText": "2/3 + 1/6 = ?", "topicName": "Fractions",
        "difficulty": 2, "correct": true, "createdAt": "2026-09-22T..."
      },
      ...
    ]
  }
}

```

`recentAttempts` returns up to 20, newest-first. `weakTopicCount` uses the same weak-topic rule as the adaptive engine.

Response 404: { "error": "Student not found" }

GET /api/quiz/next?studentId=<id>&topicId=<optional>

Returns the next question to serve, using the diagnostic-then-adaptive logic in `lib/quiz-service.ts`. Pass `topicId` (from the `/revision` page's "Practice this" button, or `?topic=` on `/practice`) to restrict to one topic and skip the diagnostic phase entirely.

File: `app/api/quiz/next/route.ts`

Response 200 (diagnostic phase):

```

{
  "question": {
    "id": "...", "text": "2/3 + 1/6 = ?",
    "options": ["5/6", "3/9", "1/2", "4/6"],
    "difficulty": 2
  },
  "topic": { "id": "...", "name": "Fractions" },
  "reason": "Diagnostic question 1 of 8 for Fractions – establishing your baseline.",
  "mode": "diagnostic",
  "questionNumber": 1,
  "totalDiagnostic": 8
}

```

Response 200 (adaptive phase): same shape, `mode`: "adaptive", `reason` is one of the "Targeted Remediation" / "Interleaved Reinforcement" / "Focused Practice" templates explaining the topic choice (see [ADAPTIVE-ENGINE.md](#)), `questionNumber` cycles `1..ADAPTIVE_SESSION_LENGTH` (15) for the progress bar.

Note: `options` never includes which index is correct — that's resolved server-side in `/api/quiz/answer` so it can never leak to the client.

Response 400: { "error": "studentId is required" }

Response 404: { "error": "No question available" } (e.g. restricted to a topic with no questions matching the target difficulty band, which shouldn't happen with the seeded data but is handled defensively)

POST /api/quiz/answer

Records an attempt, updates that topic's **Mastery** row, returns whether the answer was correct plus the student's full updated mastery snapshot.

File: `app/api/quiz/answer/route.ts`

Request body:

```
{ "studentId": "...", "questionId": "...", "selectedIdx": 2 }
```

Response 200:

```
{
  "correct": false,
  "correctIdx": 0,
  "priorScore": 0.5,
  "newScore": 0.425,
  "scoreDelta": -0.075,
  "topicName": "Fractions",
  "updatedMastery": [
    { "topicId": "...", "topicName": "Fractions", "score": 0.425 },
    ...
  ]
}
```

Response 400: missing/wrong-typed fields.

Response 404: `{ "error": "Question not found" }`

GET /api/revision?studentId=<id>

A student's weak topics with a time-to-mastery estimate for each — powers `/revision`.

File: `app/api/revision/route.ts`

Response 200:

```
{
  "weakTopics": [
    {
      "topicId": "...", "topicName": "Ratios",
      "mastery": 22, "status": "Weak", "recentAccuracy": "18%",
      "attemptsToMastery": 34
    },
    ...
  ]
}
```

Uses the same weak-topic rule as the adaptive engine (`mastery < 0.5` AND `attemptCount >= 3`); if that yields nothing, falls back to any not-yet-mastered topic (`score < 0.8`), sorted weakest first. `attemptsToMastery` comes from Model 3 (`predictAttemptsToMastery()`), given the topic's current mastery and the student's recent accuracy on it as an aptitude proxy.

GET /api/questions

Lists every question with its topic name and observed pass rate — powers the teacher [/teacher/resources](#) question-bank page.

File: `app/api/questions/route.ts`

Response 200:

```
{
  "questions": [
    {
      "id": "...", "topicId": "...", "topicName": "Fractions",
      "text": "1/2 + 1/4 = ?", "options": ["3/4", "1/6", "2/6", "1/8"],
      "answerIdx": 0, "difficulty": 1,
      "totalAttempts": 12, "passRate": 83
    },
    ...
  ]
}
```

POST /api/questions

Adds a new question to the bank.

Request body:

```
{ "topicId": "...", "text": "...", "options": ["a", "b", "c", "d"], "answerIdx": 0, "difficulty": 2 }
```

Response 201: the created [Question](#) row.

Response 400: `{ "error": "Missing required fields" }`

Response 500: `{ "error": "<message>" }` on unexpected failure.

GET /api/dashboard

Everything the teacher dashboard needs in one call: per-student mastery by topic, archetype badge, at-risk/declining-trend flags, class-wide stats, and topic-priority ranking.

File: `app/api/dashboard/route.ts`

Response 200:

```
{
  "students": [
    {
      "id": "...", "name": "Aisha Khan",
      "archetype": "On Track",
      "topicMastery": [
```

```

        { "topicId": "...", "topicName": "Fractions", "score": 0.78, "atRisk": false },
        ...
    ],
    "atRiskTopics": ["Percentages"],
    "isAtRisk": true
  },
  ...
],
"classAverage": 0.61,
"atRiskCount": 2,
"needsAttentionCount": 1,
"decliningCount": 5,
"topicPriority": [
  {
    "topicId": "...", "topicName": "Ratios",
    "avgMastery": 0.49, "pctStruggling": 0.44, "stdMastery": 0.16,
    "priority": 71.5
  },
  ...
]
}

```

archetype comes from Model 2 ([ML-MODELS.md](#)) — the mastery vector passed to it is built in a **fixed topic order** (Fractions, Ratios, Linear Equations, Percentages), because that's the order the model was trained on. **atRisk** / **atRiskTopics** / **isAtRisk** / **decliningCount** come from Model 5 ([ML-MODELS.md](#)) — a per-topic trend estimate over each student's last 10 attempts on that topic, distinct from the static mastery-threshold-based **atRiskCount** (which counts **Needs Support** archetypes). **topicPriority** comes from Model 4 ([ML-MODELS.md](#)), ranked highest-priority first.

GET `/api/dashboard/student/[id]`

One student's detail for the **teacher** drill-down view (`/teacher/students/[id]`) — a separate, richer endpoint from the student's own `/api/student/[id]/summary`, since a teacher wants the archetype label and generated insights that a student's own home page doesn't show.

File: `app/api/dashboard/student/[id]/route.ts`

Response 200:

```

{
  "name": "Priya Sharma",
  "className": "Class 8 · Mathematics",
  "status": "On Track",
  "overallMastery": 0.63,
  "questionsAttempted": 42,
  "accuracy": 0.71,
  "topicMastery": [
    { "topicId": "...", "topicName": "Fractions", "score": 0.63 },
    ...
  ],
  "recentAttempts": [
    {
      "id": "...", "topicName": "Fractions", "difficulty": 2,
      "correct": true, "createdAt": "2026-09-22T..."
    }
  ]
}

```

```

    },
    ...
  ],
  "weakTopics": [
    { "topicId": "...", "topicName": "Ratios", "mastery": 31, "status": "Weak" }
  ],
  "insights": ["..."]
}

```

`status` is the Model 2 archetype label. `insights` is a small set of data-driven observation strings (`lib/insights.ts`) — not an LLM call; it reuses Model 5's `estimateTrend()` helper against the student's own recent attempts. `recentAttempts` returns up to 30, newest-first.

Response 404: `{ "error": "Student not found" }`

6. Database

PostgreSQL via Prisma 6. (The build started on SQLite for a zero-setup first pass, then was migrated to Postgres so the data could be browsed in DBeaver like a normal relational database — see "Migration history" below.)

Schema

File: `prisma/schema.prisma`

```
model Student {
  id      String    @id @default(cuid())
  name    String
  attempts Attempt[]
  mastery Mastery[]
}

model Topic {
  id      String    @id @default(cuid())
  name    String
  questions Question[]
  mastery Mastery[]
}

model Question {
  id      String    @id @default(cuid())
  topicId String
  topic   Topic      @relation(fields: [topicId], references: [id])
  text    String
  options String[]    // 4 option strings, native Postgres array
  answerIdx Int       // index into `options` of the correct answer
  difficulty Int      // 1, 2, or 3
  attempts Attempt[]
}

model Attempt {
  id      String    @id @default(cuid())
  studentId String
  student Student    @relation(fields: [studentId], references: [id])
  questionId String
  question Question   @relation(fields: [questionId], references: [id])
  correct Boolean
  createdAt DateTime @default(now())
}

model Mastery {
  id      String    @id @default(cuid())
  studentId String
  student Student    @relation(fields: [studentId], references: [id])
  topicId String
  topic   Topic      @relation(fields: [topicId], references: [id])
  score   Float       @default(0.5) // 0-1, Elo-style running estimate

  @@unique([studentId, topicId]) // one Mastery row per (student, topic) pair
}
```

`options` was originally a JSON-stringified string, a leftover from when SQLite had no array column type. Once the app was fully on Postgres it was migrated to a native `text[]` column (see "Migrations" below) — the application code no longer does any `JSON.parse()/JSON.stringify()` at the `lib/quiz-service.ts / prisma/seed.ts` boundary; Prisma returns and accepts `options` as a plain `string[]`.

`Mastery` has exactly one row per `(studentId, topicId)` pair, enforced by the `@@unique` constraint — `lib/adaptive-engine.ts`'s `updateMastery()` result is written via `prisma.mastery.upsert()` against that same key.

Local setup options

Option A: Docker Compose (Recommended — Zero Configuration)

If you have Docker installed, this starts PostgreSQL with the credentials configured out-of-the-box:

```
docker compose up -d
```

Then copy the example env:

```
cp .env.example .env
# DATABASE_URL is already preset to: postgresql://postgres:postgres@localhost:5432/erodehack
npx prisma db push
npm run seed
```

Option B: Local Postgres (Homebrew on macOS)

```
brew install postgresql@16 # or postgresql@18
brew services start postgresql@16
createdb erodehack
```

In `.env`:

```
DATABASE_URL="postgresql://<your-macos-username>@localhost:5432/erodehack"
```

Note: Find your macOS username by running `whoami` in terminal. Homebrew Postgres uses trust auth for the local Unix user by default (no password).

```
npx prisma db push
npm run seed
```

Troubleshooting: "Please make sure to provide valid database credentials"

If Prisma outputs: `> Authentication failed against database server at 'localhost', the provided database credentials for '...' are not valid. Please make sure to provide valid database credentials for the database server at the configured address. (Prisma error P1000)`

Here is how to fix it on your machine:

1. **Check if PostgreSQL is running:** `bash pg_isready # Or on macOS Homebrew: brew services list` If it is stopped, start it: brew services start postgresql@16 or docker compose up -d`.`
2. **Verify your username & password:**
 - On macOS Homebrew: Run `whoami`. If your username is `johndoe`, your connection string must be: `DATABASE_URL="postgresql://johndoe@localhost:5432/erodehack"` (no password).
 - If using standard PostgreSQL with a `postgres` superuser password (e.g. `mypassword`): `DATABASE_URL="postgresql://postgres:mypassword@localhost:5432/erodehack"`
 - If using Docker Compose: `DATABASE_URL="postgresql://postgres:postgres@localhost:5432/erodehack"`
1. **Verify the database exists:** `bash # Create database if it does not exist: createdb erodehack # Or via psql: psql -U postgres -c "CREATE DATABASE erodehack;"` ```
2. **Test connecting with psql directly:** `bash psql "postgresql://<USER>:<PASS>@localhost:5432/erodehack" ` Once psql connects successfully, run npx prisma db push && npm run seed`.`

Connecting with DBeaver

Database -> New Database Connection -> PostgreSQL, then:

Field	Value
Host	localhost
Port	5432
Database	erodehack
Username	your macOS username
Password	*(leave blank)*

Test Connection -> Finish. You'll see `Student`, `Topic`, `Question`, `Attempt`, `Mastery` under the `public` schema.

Migrations

Directory: `prisma/migrations/` — two migrations so far:

1. `..._init` — generated by `npx prisma migrate dev --name init` against the Postgres datasource.
2. `..._question_options_native_array` — converts `Question.options` from the JSON-stringified `TEXT` column to a native `TEXT[]`. Postgres doesn't allow a subquery directly in an `ALTER COLUMN ... USING` expression, so the migration adds a new `options_new TEXT[]` column, backfills it with `ARRAY(SELECT json_array_elements_text("options")::json)` per row (which preserves option order), then drops the old column and renames the new one into place — all inside the migration's transaction, so existing seeded questions keep their data.

Check current status any time with:

```
npx prisma migrate status
```

Migration history: SQLite -> Postgres

The project was originally scaffolded with `datasource { provider = "sqlite" }` and a `prisma/dev.db` file — genuinely zero-setup, useful for the first build pass. When Postgres/DBeaver access was needed:

1. Changed `provider = "sqlite" -> provider = "postgresql"` in `schema.prisma`
2. Updated `DATABASE_URL` in `.env` to a Postgres connection string
3. Deleted the old SQLite-flavored `prisma/migrations/` (its SQL dialect doesn't carry over) and regenerated with `prisma migrate dev --name init` against the new datasource
4. Re-ran `npm run seed` and the demo-data simulation script

No application code changed — `lib/prisma.ts`'s `PrismaClient` and every query in `app/api/**` are database-engine-agnostic through Prisma.

Seeding

`prisma/seed.ts` creates the 4 fixed topics (Fractions, Ratios, Linear Equations, Percentages) and 10 questions each (3-4 per difficulty level, 1-3), with plausible (not obviously wrong) distractor options. The seed is idempotent per topic — it looks up each topic by name first and skips it (logging `Skipping <name> — already seeded`) if it already exists, so re-running `npm run seed` is safe and won't duplicate questions. To force a clean reseed anyway (e.g. after editing the question bank), wipe the tables first:

```
psql erodehack -c 'TRUNCATE "Attempt", "Mastery", "Question", "Topic", "Student" CASCADE;'
npm run seed
```